让AI智能体拥有像人类的持久记忆：基于LangGraph的长短期记忆管理实践指南

原创 腾讯程序员 腾讯技术工程 2025年12月8日 01:36 广东 620人



作者：adacyang

> 如何让AI智能体（Agent）像人类一样拥有持久的记忆，从而在复杂的连续任务中保持上下文感知和深度理解？这已成为构建高级智能体的核心挑战。本文将深入探讨Agent Memory的核心概念，并聚焦于LangGraph框架下的长短期记忆实现，详解短期会话与长期知识的存储、管理、语义检索等技巧。更进一步地，我们将通过一个引入MCP协议的实战案例，手把手带你构建一个真实的融合长记忆机制的Multi-Agent系统，直观展示中断、记忆与协作的融合。

基于大语言模型（LLM）的智能体（Agent）系统中，记忆机制是实现持续、连贯和个性化交互的核心基石，通过记忆，可以让Agent记住过往的交互，保持上下文的一致性，并能从反馈中学习，适应用户的偏好。

本文核心要点概述：

1.介绍Agent Memory的基本情况

2.LangGraph长短期记忆详解及案例说明：包含短期记忆实现、管理方法，长期记忆的实现方法，以及搭建了融合postgres数据库、集成Embedding服务进行语义搜索等可用于生产环境的真实案例。

3.引入MCP协议构建真实的Agent长记忆应用：搭建一个基于supervisor架构，集成中断机制、长短期记忆机制的multi-agent系统。

记忆机制介绍

Agent Memory是什么？



上图中（来源于Mem0[1]），左边是没有Memory的agent，右边是有Memory的agent，后者可以根据用户的过往信息（素食主义者、不喜欢乳制品）给出更合理的响应（不含乳制品的素食菜单），而前者的回答显然是不合适的。

简单来说，Memory是赋予Agent记忆能力的技术和架构，能够让Agent像人一样记住过去的交互、学到的知识、执行过的任务及未来的计划，是将一个LLM转变为能够执行复杂、长期任务的真正”智能体“的核心所在。

关于Agent Memory我们需要考虑什么？

如何获取记忆：通过和用户交互、环境交互...

怎么组织记忆：模型参数、模型上下文、数据库

怎么利用记忆：RAG、Few-shot...

有哪些Memory类型？

关于Memory的分类，有许多种分类体系，这里我们只讨论最常见及最易于理解的。

正如人类利用长短期记忆进行有效的交互和学习一样，Agent的记忆机制通常划分为短期记忆（short-term memory）和长期记忆(long-term memory)，短期记忆决定了Agent在微观任务上的即时表现，而长期记忆则作为持久知识库，决定了Agent在宏观时间尺度上的智能深度和个性化水平，通过两者配合，Agent才能表现出连贯性、上下文感知能力，才会显得更智能。

Agent Memory如何工作？

Agent通常通过以下几步来有效地管理记忆，使得每次于用户的交互都更加精准智能：

1. 记忆存储：通过设计一系列策略来存储重要的交互信息，这些信息可能来源于对话内容、历史数据或任务要求等等。
2. 记忆更新：记忆会随着交互的发生，不断地进行更新，例如用户的偏好、最新的近况等等。记忆更新使得Agent能够不断优化其响应。

3. 记忆检索：Agent根据当下的需求，去记忆中检索需要的记忆内容，从而提供更加智能的回复。

Agent Memory怎么实现？

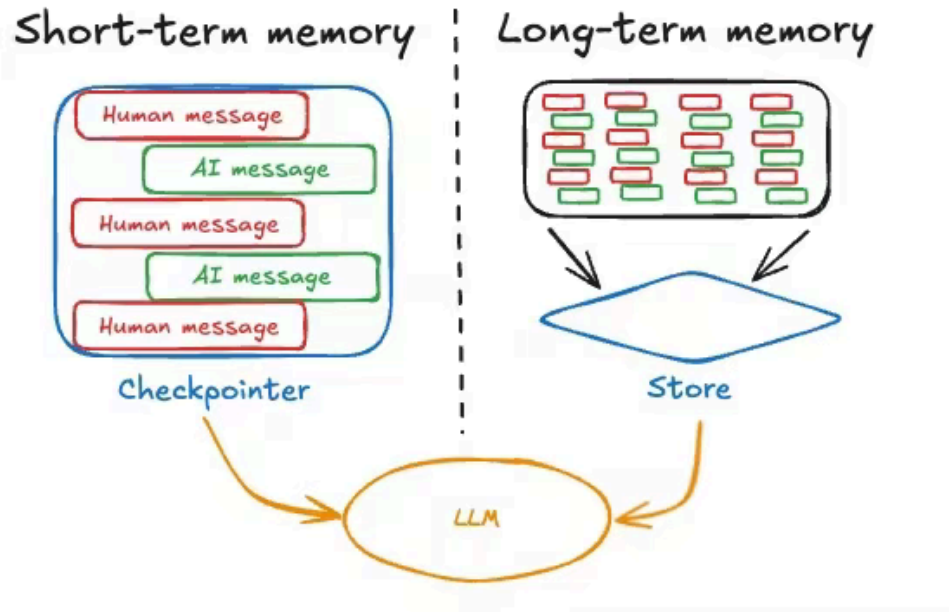
1. 物理外挂：即外置数据库和 RAG，需要检索当前query相关的内容，例如：Mem0、ACE。好处是即插即用，坏处是不够end-to-end
2. Memory as Reasoning / Tool：通过训练Reasoning或Tool的方式动态更新context，例如：MemAgent、memory-R1。好处是更接近end-to-end，但不是很灵活。
3. 参数更新：LLM本身就是一个Memory体，所有参数都是它的Memory，通过更新参数来更新记忆，这种方式是最本质的，但也是最难实现的。

LangGraph中的记忆管理

LangGraph[2]作为一款面向多智能体协作与状态管理的框架，其设计了巧妙的记忆管理系统，旨在为Agent提供在不同交互中存储、检索和利用信息的能力。它区分了两种主要的记忆类型：短期记忆和长期记忆。在实际使用中，通过这两种记忆协同，既能保障实时任务的高效执行，又支持了跨任务、跨周期的经验复用。

• short-term memory（通过Checkpointer实现）：针对单个对话线程，核心价值在于保障对话的临时性，使得Agent能够跟踪会话中的多轮对话，可以在该线程内的任何时刻被回忆。

• long-term memory（通过Store实现）：可以跨对话线程共享，可以在任何时间，任何线程中被回忆，而不像短期记忆局限于单个对话。



通过下表，可以更清晰的看到两者的区别：

	short-term memory	long-term memory
目的	维持对话上下文	存储跨会话的持久化事实、偏好和知识
持久性	会话级别（可以临时，可以持久）	应用级别（始终持久）
作用域	单一会话	跨会话、跨用户
持久化方式	检查点（checkpoint）	存储（Store）
更新机制	自动（在每个图步骤后保存状态）	手动/显式
典型用途	对话历史、中间状态	用户偏好、知识库、语义记忆

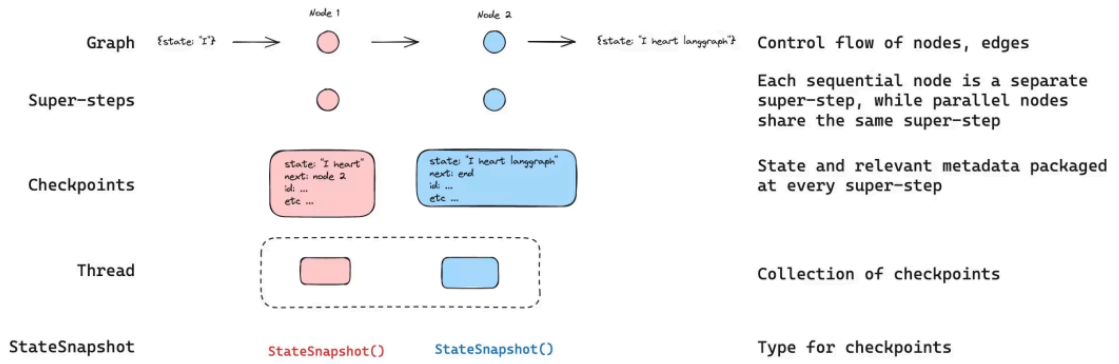
LangGraph记忆的架构基础

要想更好的理解LangGraph中的记忆机制，首先需要理解其支持双轨记忆系统的核心概念。

Checkpointer

LangGraph有一个内置的持久化（Persistence）层，通过checkpointer实现，能够持久化存储图状态，这使得开发记忆功能和人类干预功能成为可能。

当使用检查点编译一个图时，检查点会在每个super-step保存图状态的checkpoint，这些checkpoint被保存到一个thread中，可以在图执行后访问。因为threads允许在执行后访问图的状态，所以可以实现记忆、人机协作、时间旅行、容错等多种强大的功能。



工作流程：

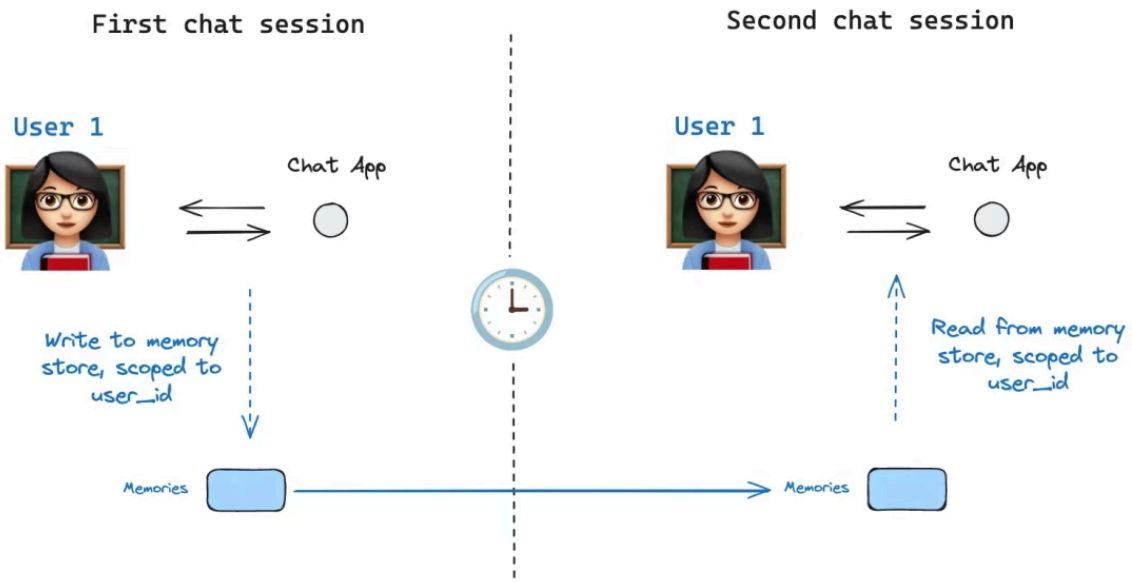
```
用户输入 → [节点 1] → 💾 保存状态 → [节点 2] → 💾 保存状态 → 输出
↓ ↓
Checkpoint 1   Checkpoint 2
```

Thread

为了管理多个独立的对话，LangGraph使用了thread的概念。thread_id是由checkpointer保存的每个checkpoint的唯一id，是激活和区分不同对话线程的唯一key。在调用图的invoke或stream方法时，通过configurable字典传入一个thread_id，就代表这次操作属于thread_id这个特定的对话。

Store

如上所述，图状态可以由checkpointer在每个super-step写入线程，从而实现状态的持久化。但是，如果想在多个线程之间保留一些信息的话，那么就需要用到Store。Store本质上是一个暴露给图节点和工具的键值数据库，与checkpointer的自动化快照不同，Store需要显式和主动的进行操作。



Namespace

Store中的数据通常通过更持久的标识来组织。user_id是最常见的，用于关联用户的所有信息，此外，namespace提供了一种数据隔离机制，例如，使用使用 (“memories”, user_id) 这样的元组作为命名空间，可以将用户的记忆与其他类型的数据（如用户偏好 (“preferences”, user_id)）清晰地分离开来，避免数据冲突，保持知识库的整洁有序。

短期记忆详解

InMemorySaver内存会话临时存储

对于开发、原型设计和测试阶段，最简单快捷的方式是使用InMemorySaver。它将所有的对话状态存储在内存中的一个Python字典里。

1.设置记忆管理检查点

```
from langchain_openai import ChatOpenAI
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.prebuilt import create_react_agent

# 初始化检查点保存器
checkpointer = InMemorySaver()
```

2.定义大模型并创建agent

```
```

```
BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

agent = create_react_agent(
    model=model,
    tools=[],
    # 传入检查点，是将持久化能力“注入”图的关键步骤。编译后的graph对象现在具备了状态管理的能力。
    checkpointer=checkpointer
)
```

如果是底层自定义api在图构建阶段传入检查点的代码是graph = builder.compile(checkpointer=checkpointer)。

3.短期记忆-内存后端

```
config = {"configurable": {"thread_id": "1"}}  # 激活记忆机制的核心。如果没有提供thread_id，每

response = agent.invoke(
    {"messages": [{"role": "user", "content": "你好，我叫ada！"}]},
    config
)

print(f"thread1_bot_answer: {response['messages'][-1].content}")

response = agent.invoke(
    {"messages": [{"role": "user", "content": "你好，请问你还记得我叫什么名字么？"}]},
    config
)

print('------------线程1-----------------')
print(f"thread1_bot_answer: {response['messages'][-1].content}")

new_config = {"configurable": {"thread_id": "2"}}
response = agent.invoke(
    {"messages": [{"role": "user", "content": "你好，请问你还记得我叫什么名字么？"}]},
    new_config
)
```

```
print('------------线程2------------------')
print(f"thread2_bot_answer: {response['messages'][-1].content}")
```

执行上面代码，可以看到输出如下：

```
thread1_bot_answer: 你好，Ada！很高兴认识你！😊

这是一个很美的名字呢！有什么我可以帮助你的吗？无论是想聊聊天，还是有任何问题需要解答，我都很乐意为你提供
------------线程1------------------
thread1_bot_answer: 当然记得！你刚才告诉我你叫 Ada～很高兴再次和你打招呼！😊
------------线程2------------------
thread2_bot_answer: 你好！很抱歉，我无法记住之前对话中的个人信息，比如你的名字。这是为了保护你的隐私，
```

短期记忆与线程相关，在对话时，需要在配置中传入thread_id。通过上面的结果我们可以看到，当我们传入相同的thread_id时，agent就可以记住用户的名字，然而当我们更换thread_id时，agent就不记得用户的名字了。

需要注意的是，**InMemorySaver将所有状态都保存在内存中**，一旦程序终止，那么所有对话历史都会消失。

数据库持久化存储

可以发现，上面一小节的代码在应用程序结束后再启动，记忆就又消失了。这是因为InMemorySaver仅仅是把记忆保存在内存中，应用程序结束后释放内存记忆就消失了。在生产环境中常常使用数据库支持的检查点记录器持久化保存记忆，以保证数据的可靠性和服务的连续性。

这里我们以postgres数据库为例来说明，怎么持久化地保存记忆数据。

1.首先安装以下依赖：

```
pip install -U "psycopg[binary,pool]" langgraph-checkpoint-postgres
```

2.安装postgres数据库，具体的安装方法可以参考：**Linux下安装PostgreSQL_linux安装postgresql-CSDN博客**。这里选择以源码的方式进行安装，安装包从官网（**PostgreSQL: Downloads**）下载，选择最新的postgresql-18.0.tar.gz。

3.安装数据库成功后，编码如下代码。

DB_URI是数据库连接的URL。想要自动保存在数据库中的State需要在PostgresSaver.from_conn_string(DB_URI)上下文中操作。

```
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START
```

```
from langgraph.checkpoint.postgres import PostgresSaver

BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()  # 第一次调用时必须要setup()

    def call_model(state: MessagesState):
   response = model.invoke(state["messages"])
   return {"messages": response}

    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")

    graph = builder.compile(checkpointer=checkpointer)

    config = {
   "configurable": {
"thread_id": "1"
   }
    }

    response = graph.invoke(
   {"messages": [{"role": "user", "content": "你好，我叫ada！"}]},
   config
    )

    print(response['messages'][-1].content)

    response = graph.invoke(
   {"messages": [{"role": "user", "content": "你好，请问你还记得我叫什么名字么？"}]},
   config
    )

    print(response['messages'][-1].content)
```

运行一次上述代码后，关闭应用程序后重启，再次运行上述代码，print结果如下：

```
bot_answer_1: 你好，Ada！很高兴再次见到你！😊

你的名字真动听！今天有什么我可以帮你解答或者想聊的话题吗？
bot_answer_2: 当然记得！你告诉我你叫 **Ada**。很高兴再次和你打招呼！😊
```

可以看到，记忆已经被保存了。我们检查数据库可以发现，postgres数据库中出现了四个表：

```
postgres=# \dt
                List of tables
 Schema |          Name           | Type  |  Owner
--------+-------------------------+-------+----------
 public | checkpoint_blobs        | table | postgres
 public | checkpoint_migrations   | table | postgres
 public | checkpoint_writes       | table | postgres
 public | checkpoints             | table | postgres
(4 rows)
```

上述表中，checkpoints表是”状态快照“表，每当程序执行一个step时，它就会在这张表中创建一条新记录，这条记录就是一个检查点的快照。查询该表，可以得到如下结果：

```
postgres=# select * from checkpoints limit 5;
 thread_id | checkpoint_ns |           checkpoint_id              |        parent_checkpoint_id         | type |
                                                
```

列名	含义	举例说明
thread_id	线程ID	上表中，所有thread_id都为1，表示这些记录都属于同一个会话流
checkpoint_ns	检查点命名空间（Namespace），用于对检查点进行分组或分类	上表中都是空的，表示未使用或使用了默认的命名空间
checkpoint_id	检查点的唯一标识符，该记录的主键	
parent_checkpoint_id	父检查点的ID，它将检查点链接起来	第一条记录的parent_checkpoint_id是空的，代表是整个流程的起点。
checkpoint	核心状态数据，是一个json对象。	ts代表时间戳；channel_values代表通道值，可以理解为工作流中的变量值；updated_channels代表在当前这步中被修改过的通道
metadata	该检查点本身的元数据	step表示这是工作流的第几步；source表来源，"input"指外外部输入，"loop"指工作流内部循环或某个节点执行的结果

理解了上面checkpoints表后，那么不禁会问，真正的消息内容被存到了哪里呢？真正的消息内容存储在checkpoint_writes表中，如下：

```
postgres=# SELECT thread_id, channel, checkpoint_id,blob  FROM checkpoint_writes WHERE channel = 'messages';
 thread_id | channel  |            checkpoint_id             |

                                                
```

```
from langgraph.graph import START, StateGraph
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict

class State(TypedDict):
    foo: str

# 子图
def subgraph_node_1(state: State):
    return {"foo": state["foo"] + "bar"}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()

# 父图
builder = StateGraph(State)
builder.add_node("node_1", subgraph)
builder.add_edge(START, "node_1")

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)
```

如果子图希望使用自己的短期记忆，那么需要在编译子图时，显示传入子图的检查点：

```
subgraph_builder = StateGraph(...)
subgraph = subgraph_builder.compile(checkpointer=True)
```

工具中的记忆交互

在工具中读取状态：

LangGraph允许工具直接访问和读取当前的图状态，使其具备上下文感知能力。

核心机制：state: Annotated[CustomState, InjectedState]，InjectedState的作用是在调用这个工具时，将当前的完整状态对象作为第一个参数传递到工具中，使得这个工具能根据当前状态来执行更智能的操作。

```
from typing import Annotated
from langchain.chat_models import init_chat_model
from langgraph.prebuilt import InjectedState, create_react_agent
from langgraph.prebuilt.chat_agent_executor import AgentState

BASE_URL=""
```

```
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

class CustomState(AgentState):
    user_id: str

def get_user_info(
    state: Annotated[CustomState, InjectedState]
) -> str:
    """查询用户信息"""
    user_id = state["user_id"]
    return"ada"if state["user_id"] == "user_ada"else"Unknown"

agent = create_react_agent(
    model=model,
    tools=[get_user_info],
    state_schema=CustomState,
)

agent.invoke({
    "messages": "查询用户信息",
    "user_id": "user_ada"
})
```

返回结果如下：

```
根据查询结果，您的用户信息显示用户名为：**ada**\n\n这是您的用户信息查询结果。如果您需要了解其他信息或有
```

在工具中写入状态：

如果要在工具执行期间修改图的记忆，那么可以直接从工具返回状态更新。这对于持久化中间结果、传递信息给后续工具等非常有用。

核心机制：工具返回Command对象。此时，LangGraph会将其返回值解释为对状态的直接修改指令。Command(update={...})中的字典定义了要更新的状态字段及其新值。这允许工具在完成其主要任务的同时，将结果写回智能体的短期记忆中，从而影响后续的决策。

```
```

```python
from typing import Annotated
from langchain_core.tools import InjectedToolCallId
from langchain_core.runnables import RunnableConfig
from langchain_core.messages import ToolMessage
from langgraph.prebuilt import InjectedState, create_react_agent
from langgraph.prebuilt.chat_agent_executor import AgentState
from langgraph.types import Command

class CustomState(AgentState):
    user_name: str

def update_user_info(
    tool_call_id: Annotated[str, InjectedToolCallId],
    config: RunnableConfig
) -> Command:
    """查询并更新用户信息"""
    user_id = config["configurable"].get("user_id")
    name = "ada"if user_id == "user_123"else"Unknown user"
    return Command(update={
   "user_name": name,
   # 更新消息历史记录
   "messages": [
ToolMessage(
    "成功查询到用户信息",
    tool_call_id=tool_call_id
)
   ]
    })

def greet(
    state: Annotated[CustomState, InjectedState]
) -> str:
    """找到用户信息后，使用此方式向用户问好。"""
    user_name = state["user_name"]
    return f"你好 {user_name}! "

agent = create_react_agent(
    model=model,
    tools=[update_user_info, greet],
    state_schema=CustomState
)

agent.invoke(
    {"messages": [{"role": "user", "content": "向用户打招呼"}]},
    config={"configurable": {"user_id": "user_123"}}
)
```

输出结果如下：



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

长期记忆详解

LangGraph中的长期记忆允许系统在不同对话中保留信息，是跨对话线程共享的，可以在任何时间、任何线程中被回忆。与短期记忆不同，长期记忆保存在自定义的命名空间中，每个记忆都组织在一个自定义的namespace和一个唯一的key下。

记忆存储：LangGraph将长期记忆存储为JSON文档，使用Store进行管理，允许存储结构化和非结构化的数据。

记忆更新时机：

- 热路径（Hot Path）：在应用程序逻辑运行时实时创建记忆（store.put()），优点是实时更新，但可能增加程序复杂性、延迟等问题。

- 后台（Background）：作为单独的异步任务创建记忆（store.put()），优点是避免主应用延迟、逻辑分离，难点在于确定更新频率和触发时机。

记忆检索：

- store.get()：根据命名空间和键精确获取记忆。

- store.search()：在指定命名空间内实现灵活记忆检索，不但可以通过命名空间和标识符，更可以通过语义检索到记忆内容。通常需要Store配置一个embed来支持语义搜索。

记忆的应用：

- 语义记忆：存储事实和概念。分为以下两种情况：Profile：将关于用户、组织或代理自身的特定信息存储为一个持续更新的JSON文档，需要模型来生成新的Profile或更新已有JSON档案；Collection：将记忆存储为一组独立的文档，易于生成，但检索和更新较为复杂，且可能

难以捕获记忆间的完整上下文。在应用时，可以将检索到的记忆作为上下文或系统指令的一部分传递给LLM，用于个性化响应和回答事实性问题。

- 情景记忆：存储过去的事件或行为经验。通常通过few-shot example prompt来实现，以指导模型完成任务。

- 程序记忆：存储执行任务的规则或指令。通常通过修改代码自身的prompt来实现，将其应用于LLM。

InMemoryStore

与Checkpointer类似，InMemoryStore用于快速开发和原型验证。它将所有数据存储在内存中。

```
from langchain.chat_models import init_chat_model
from langchain_core.runnables import RunnableConfig
from langgraph.store.memory import InMemoryStore
from langgraph.config import get_store
from langgraph.prebuilt import create_react_agent

store = InMemoryStore()

BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

store.put(
    ("users",),  # 命名空间：元组类型，类比文件系统中的文件夹，支持分层组织结构
    "user_123",  # 键：字符串，是命名空间内的唯一标识符，一般推荐使用uuid库生成唯一标识符
    {
   "name": "ada",
   "language": "中文",
    }  # 值：Python字典类型，比如保存公共角色资料时可以是包含姓名、偏好等键值对的字典
)

def get_user_info(config: RunnableConfig) -> str:
    """查找用户信息的函数，可以查看长期记忆中储存的用户信息"""
    store = get_store()  # 获取上下文中可用的store实例
    user_id = config["configurable"].get("user_id")
```

```
    user_info = store.get(("users",), user_id)  # 输入命名空间和键进行精确查询
    return str(user_info.value) if user_info else"Unknown user"

agent = create_react_agent(
    model=model,
    tools=[get_user_info],
    # 传入store
    store=store
)

response = agent.invoke(
    {"messages": [{"role": "user", "content": "帮我查找长期记忆中储存的用户信息"}]},
    config={"configurable": {"user_id": "user_123"}}
)

print(response['messages'])
```

输出结果如下，可以看到在工具函数中成功调用store查找到了保存的用户信息：



数据库持久化存储

为了让记忆真正"长期"，生产环境必须使用数据库支持的Store，LangGraph目前主要支持PostgresStore和RedisStore。我们以PostgresStore为例来进行说明。

1. 首先，先安装对应的包：

```
pip install -U "psycopg[binary,pool]" langgraph-checkpoint-postgres
```

2. Postgres数据库的安装，请参考上文。

3. 接下来进行示例说明。整体过程与Checkpointer类似，关键区别在于Store是怎样在节点内部被访问和使用的。

```
import uuid
from langchain.chat_models import init_chat_model
from langchain_core.runnables import RunnableConfig
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.store.base import BaseStore
from langgraph.store.postgres import PostgresStore
from langgraph.checkpoint.postgres import PostgresSaver
```

```
BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

DB_URI = "postgresql://postgres:postgres@localhost:5432/postgres?sslmode=disable"

with (
    PostgresStore.from_conn_string(DB_URI) as store,
    PostgresSaver.from_conn_string(DB_URI) as checkpointer,
):
    store.setup()  # 第一次调用时必须要setup()
#checkpointer.setup()

    # 声明store参数
    def call_model(
   state: MessagesState,
   config: RunnableConfig,
   *,
   store: BaseStore,  # 在节点中访问store的标准方式，需要在函数签名上，加一个store
    ):
   # 从store中读取记忆
   user_id = config["configurable"]["user_id"]
   namespace = ("memories", user_id)
   memories = store.search(namespace, query=str(state["messages"][-1].content))
   info = "\n".join([d.value["data"] for d in memories])
   system_msg = f"你是一个与人类交流的小助手，用户信息：{info}"

   # 向store中写入记忆
   last_message = state["messages"][-1]
   if"记住"in last_message.content.lower():
memory = "用户名字是ada"
store.put(namespace, str(uuid.uuid4()), {"data": memory})

   response = model.invoke([{"role": "system", "content": system_msg}] + state["messages"
   return {"messages": response}

    builder = StateGraph(MessagesState)
    builder.add_node(call_model)
    builder.add_edge(START, "call_model")
```

```
    graph = builder.compile(checkpointer=checkpointer, store=store)  # agent同时配备了短期记

    # 第一次对话，告诉agent用户的名字
    config = {
   "configurable": {
"thread_id": "3",
"user_id": "1",
   }
    }

    response = graph.invoke(
   {"messages": [{"role": "user", "content": "你好，我叫ada！记住这个名字呦~"}]},
   config
    )

    print(response['messages'][-1].content)

    # 第二次对话，新线程，询问agent记不记得用户的名字
    config = {
   "configurable": {
"thread_id": "4",
"user_id": "1",
   }
    }

    response = graph.invoke(
   {"messages": [{"role": "user", "content": "我的名字是什么?"}]},
   config
    )
    print(response['messages'][-1].content)
```

输出结果如下：

```
# ------------------第一次对话----------------------
你好ada！很高兴认识你～我已经记住你的名字啦！✨

有什么我可以帮你的吗?
# ------------------第二次对话----------------------
你的名字是ada
```

在第一次对话时，对话线程id为3，agent被要求记住用户的名字，并且根据代码逻辑，用户名字信息是通过store.put()写入数据库的。第二次对话时，线程id为4，当被问起用户的名字时，agent通过store.search()办法从数据库中检索到了这个信息，并成功回答，这展示了Store的跨记忆存储能力。

长期知识赋能工具

在工具中读取长期记忆：

参考上文中，长期记忆-InMemoryStore中的示例。

其中，核心在于store = get_store()，这个函数是一个上下文感知的辅助函数，能够在工具执行时，自动获取并返回compile或create_react_agent中传入的store实例。

在工具中写入长期记忆：

```
from typing_extensions import TypedDict

from langchain.chat_models import init_chat_model
from langgraph.config import get_store
from langchain_core.runnables import RunnableConfig
from langgraph.prebuilt import create_react_agent
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

class UserInfo(TypedDict):
    name: str

def save_user_info(user_info: UserInfo, config: RunnableConfig) -> str:
    """将用户信息保存到store"""
    store = get_store()
    user_id = config["configurable"].get("user_id")
    store.put(("users",), user_id, user_info)
    return"成功保存了用户信息"

agent = create_react_agent(
    model=model,
    tools=[save_user_info],
    store=store
)
```

```
agent.invoke(
    {"messages": [{"role": "user", "content": "我叫ada! 请你记住我的名字"}]},
    config={"configurable": {"user_id": "user_123"}}
)
```

首先需要定义要存储的数据内容，即UserInfo，它为LLM提供了一个清晰的结构化输出格式，当LLM决定调用save_user_info的工具时，会自动生成一个包含name字段的字典。然后调用store.put()方法，将数据存储下来。

```
{'messages': [HumanMessage(content='我叫ada! 请你记住我的名字', additional_kwargs={}, response_metadata={}, id='a9c328af-f469-4fd8-852e-786b967b0b8
b'),
  AIMessage(content='', additional_kwargs={'tool_calls': [{'id': 'call_89f8460e9fc04f2698f9bde9', 'function': {'arguments': '{"user_info": {"nam
e": "ada"}}', 'name': 'save_user_info'}, 'type': 'function', 'index': -1}], 'refusal': None}, response_metadata={'token_usage': {'completion_token
s': 20, 'prompt_tokens': 180, 'total_tokens': 200, 'completion_tokens_details': None, 'prompt_tokens_details': None}, 'model_name': 'default', 'sy
stem_fingerprint': None, 'id': '46c5f690e90b4e2a96049d50062c6e92', 'service_tier': None, 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_
run--f718a7b8-937a-42de-bd47-5826682ab783-0', tool_calls=[{'name': 'save_user_info', 'args': {'user_info': {'name': 'ada'}}, 'id': 'call_89f8460e9
fc04f2698f9bde9', 'type': 'tool_call'}], usage_metadata={'input_tokens': 180, 'output_tokens': 20, 'total_tokens': 200, 'input_token_details': {},
'output_token_details': {}}),
  ToolMessage(content='成功保存了用户信息', name='save_user_info', id='9c3fcd0d-ea04-4807-9340-54816648ed31', tool_call_id='call_89f8460e9fc04f2698f
9bde9'),
  AIMessage(content='好的，ada！我已经记住了你的名字。有什么我可以帮助你的吗？', additional_kwargs={'refusal': None}, response_metadata={'token_usage':
{'completion_tokens': 16, 'prompt_tokens': 208, 'total_tokens': 224, 'completion_tokens_details': None, 'prompt_tokens_details': None}, 'model_nam
e': 'default', 'system_fingerprint': None, 'id': '5da7067d7b5c43ee9541f852a5caa1e0', 'service_tier': None, 'finish_reason': 'stop', 'logprobs': No
ne}, id='lc_run--28ab3570-971a-4436-87b3-1c917db4b139-0', usage_metadata={'input_tokens': 208, 'output_tokens': 16, 'total_tokens': 224, 'input_to
ken_details': {}, 'output_token_details': {}})]}
```

```
store.get(("users",), "user_123").value

# 输出: {'name': 'ada'}
```

语义搜索

Store最强大的功能之一是支持语义搜索，这能将Store从一个简单的键值数据库，转变为一个功能完备的向量数据库。智能体不再只能通过精确的关键词来检索记忆，而是能够根据概念的相似性来查找相关信息。**这实际就是一套RAG流程**。

下面我们将基于自定义部署的Embedding服务，来演示如何进行长期记忆语义搜索。特别说明的是，代码仅供演示使用，实际使用可以参考下面代码，编写更规范的代码。

1.首先我们需要自己**部署一个Embedding服务**，这里我们以Qwen3-Embedding-4B为例。

2.**创建自定义Embedding类**，这个类需要继承自langchain.embeddings.base.Embeddings，这个类的作用是负责与Embedding服务进行通信。

```
import requests
from typing import List, Optional, Dict
from langchain.embeddings.base import Embeddings

class SelfAPIEmbeddings(Embeddings):
    """
    一个自定义的 Embedding 类，用于调用自部署的 embedding API 服务。
    """
    def __init__(self):
   """
```

```
    初始化函数。
    """
    self.token = ""
    self.url = ""
    self.model_id = ""

     def _call_api(self, texts: List[str]) -> List[List[float]]:
    """
    内部方法，用于调用 API。
    *** 您需要根据您自己服务的实际 API 格式来修改这部分 ***
    """
    try:
payload = {
     'model': self.model_id,
     'input': texts
}

headers = {
     'Content-Type': 'application/json',
     'Authorization': f'Bearer {self.token}'
}

response = requests.post(self.url, headers=headers, data=json.dumps(payload))

# 判断是否异常
if response.status_code != 200:
     print(response.json())
     exit()

res = response.json()
return res['data']
    except requests.exceptions.RequestException as e:
print(f"Error calling embedding API: {e}")
# 可以选择返回空列表或重新抛出异常
raise e

     def embed_documents(self, texts: List[str]) -> List[List[float]]:
    """
    为文档列表生成 embeddings。
    """
    if not texts:
return []
    # 为了避免请求体过大，可以分批处理
    # 这里为了简单起见，一次性发送所有文本
    new_texts = []
    for i in texts:
i = eval(i)
```

```
new_texts.append(i['text'])

    res = self._call_api(new_texts)
    new_res = []
    for i in res:
new_res.append(i['embedding'])
    return new_res

     def embed_query(self, text: str) -> List[float]:
    """
    为单个查询文本生成 embedding。
    """
    if not text:
return []
    result = self._call_api([text])
    return result[0]['embedding']
```

3.**将自定义的Embedding类集成到工作流中**，通过在Store中配置index来启用语义搜索。

```
from langchain.embeddings import init_embeddings
from langgraph.store.memory import InMemoryStore

custom_embeddings = SelfAPIEmbeddings()
store = InMemoryStore(
     index={
    "embed": custom_embeddings,
    "dims": 2560,
     }
)
```

4.**语义查询测试。**

```
store.put(("user_123", "memories"), "1", {"text": "我喜欢吃披萨"})
store.put(("user_123", "memories"), "2", {"text": "我是一名程序员"})

items = store.search(
    ("user_123", "memories"), query="我肚子饿了", limit=1
)
```

输出如下，尽管查询没有”披萨“这个词，但是通过Embedding模型计算，知道披萨和饿了是相近的语义，因此成功检索出了相关的记忆。

```
```

```
[Item(namespace=['user_123', 'memories'], key='1', value={'text': '我喜欢吃披萨'}, created_
```




腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

短期记忆管理策略

随着对话的进行，短期记忆（对话历史）会不断增长，可能会超出LLM的上下文窗口，导致请求调用失败，或者使LLM反应变慢、变差。这时，就需要对记忆进行管理了。常见的解决办法有：

- 修剪消息（trim messages）：移除前 N 条或后 N 条消息（在调用 LLM 之前）。最简单直接，但信息丢失严重，适合短期任务、无状态问答机器人、近期上下文最重要的应用。

- 删除消息（delete messages）：从LangGraph状态中永久删除消息。可以精确的控制移除内容，但需要自定义逻辑来判断哪些消息需要删除，适合用于移除不再需要的冗余系统消息、工具输出或错误信息。

- 总结消息（summarize messages）：汇总历史记录中的早期消息并将其替换为摘要。保留了核心语义信息，但计算成本高，实现相对复杂，适合用于长期连续对话、需要维持深度长期上下文的智能体。

- 自定义策略：例如消息过滤等。

修剪消息

管理对话历史的一个重要概念是限制传递给模型的消息数量，trim_messages就是LangChain提供的一个实用函数，它根据指定的策略、token限制、模型要求以及是否包含系统消息等来裁剪消息列表，它的主要目的是确保对话历史不会超出模型的上下文窗口大小。

它的解决策略是：当消息历史过长时，从开头或结尾丢弃一部分消息，以确保总长度符合限制。

```
from langchain_core.messages.utils import (
    trim_messages,
    count_tokens_approximately
)
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph, START, MessagesState

BASE_URL=""
TOKEN=""
```

```
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

summarization_model = model.bind(max_tokens=128)

def call_model(state: MessagesState):
    # 保留最近消息，总 token ≤ 128
    messages = trim_messages(
   state["messages"],
   strategy="last",  # 保留最后的消息
   token_counter=count_tokens_approximately,
   max_tokens=128,
   start_on="human",  # 确保第一条消息（不包括系统消息）是从human消息开始保留
   end_on=("human", "tool"),  # 保留到human或tool消息为止
   allow_partial=False,  # 不允许分割消息内容
   include_system=True  # 保留system prompt
    )
    # --- 在这里打印传入模型的内容 ---
    print("-" * 20)
    print(f"Messages being sent to the model (trimmed to <= 128 tokens): {len(messages)}"
    for msg in messages:
   print(f"  [{msg.type.upper()}]: {msg.content}")
    print("-" * 20)
    response = model.invoke(messages)
    return {"messages": [response]}

checkpointer = InMemorySaver()
builder = StateGraph(MessagesState)
builder.add_node(call_model)
builder.add_edge(START, "call_model")
graph = builder.compile(checkpointer=checkpointer)

# 如果需要在agent中修剪，那么需要将pre_model_hook和trim_messages结合使用
"""
def call_model(state)
    messages = trim_messages(...)

agent = create_react_agent(
 model,
 tools,
 pre_model_hook=call_model,
```

```
  checkpointer=checkpointer,
)
"""
```

发起请求如下：

```
config = {"configurable": {"thread_id": "1"}}

graph.invoke({"messages": "你好，我叫ada"}, config)
graph.invoke({"messages": "请写一首诗，关于小狗的"}, config)
graph.invoke({"messages": "再写一首关于小猫的"}, config)
final_response = graph.invoke({"messages": "我叫什么名字呢？"}, config)
```

输出打印如下：

```
--------------------
Messages being sent to the model (trimmed to <= 128 tokens): 1
  [HUMAN]: 你好，我叫ada
--------------------


--------------------
Messages being sent to the model (trimmed to <= 128 tokens): 3
  [HUMAN]: 你好，我叫ada
  [AI]: 你好，Ada！很高兴认识你！😊

这是一个很美的名字呢！有什么我可以帮助你的吗？无论是想聊天、有问题需要解答，还是需要任何形式的帮助，我都
  [HUMAN]: 请写一首诗，关于小狗的
--------------------


--------------------
Messages being sent to the model (trimmed to <= 128 tokens): 5
  [HUMAN]: 你好，我叫ada
  [AI]: 你好，Ada！很高兴认识你！😊

这是一个很美的名字呢！有什么我可以帮助你的吗？无论是想聊天、有问题需要解答，还是需要任何形式的帮助，我都
  [HUMAN]: 请写一首诗，关于小狗的
  [AI]: 好的，Ada！为你写一首关于小狗的可爱小诗，希望你喜欢：

**《小狗的约定》**
.....  （此处省略诗的内容）
这首诗捕捉了小狗的活泼、忠诚和它们带给我们的温暖。你觉得怎么样？😊
  [HUMAN]: 再写一首关于小猫的
--------------------


--------------------
Messages being sent to the model (trimmed to <= 128 tokens): 3
```

```
  [HUMAN]: 再写一首关于小猫的
  [AI]: 好的，Ada！这首关于小猫的诗，希望同样能带给你一丝轻盈与温柔：

**《小猫的遐想》**
.....   （此处省略诗的内容）
希望你喜欢这首小诗！🐾
  [HUMAN]: 我叫什么名字呢?
--------------------
```

模型最终的输出为：

```
final_response["messages"][-1].pretty_print()
================================== Ai Message ==================================

你并没有告诉我你的名字呢～😊

在对话开始时，你只说了“再写一首关于小猫的”，所以我现在还不知道你的名字。如果你愿意告诉我，我很乐意记住它，并在以后的对话中称呼你！✨

或者，如果你想要一个临时的昵称，我可以叫你：
- **小橘**（像橘猫一样温暖）
- **雪球**（像白色小猫的毛茸茸）
- **爪爪**（来自小猫的肉垫）
- 或者其他你喜欢的名字～

请告诉我你希望我怎么称呼你吧！🐾
```

可以看出，传递给模型的消息内容已经被裁剪，修剪的过程为：

1.保留系统消息，include_system=True

2.strategy="last"，反转消息列表，以便从最新的消息开始处理

3.累积token数量，当达到max_tokens限制，那么进行修剪

4.修剪时，由于allow_partial=False，因此，保留的消息都是完整的；且start_on="human"，所以修剪后第一条非system prompt是用户消息

虽然对传递给模型的历史消息进行了裁剪，但是查询state可以发现，历史记录仍被完整的保留在内存中，没有被删除。

```python
print("\n" + "="*30)
print("  查看 thread_id='1' 的完整对话历史")
print("="*30)

current_state = graph.get_state(config)
conversation_history = current_state.values["messages"]

for message in conversation_history:
    print(f"[{message.type.upper()}]: {message.content}")
```

删除消息

这种方法允许从状态中永久移除特定的消息。要删除消息，不能直接从状态的messages列表中移除，而是使用**RemoveMessage**函数，从graph state中直接删除消息来管理对话历史。

为了让RemoveMessage生效，需要使用带有add_messages reducer的状态键，例如MessagesState。

删除特定消息：

```python
from langchain_core.messages import RemoveMessage
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph, START, MessagesState

BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)

def delete_messages(state):
    messages = state["messages"]
    if len(messages) > 2:
   # 删除最早的两条消息
   return {"messages": [RemoveMessage(id=m.id) for m in messages[:2]]}

def call_model(state: MessagesState):
    response = model.invoke(state["messages"])
    return {"messages": response}

builder = StateGraph(MessagesState)
builder.add_sequence([call_model, delete_messages])
builder.add_edge(START, "call_model")

checkpointer = InMemorySaver()
app = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "1"}}

for event in app.stream(
    {"messages": [{"role": "user", "content": "你好，我是ada"}]},
    config,
    stream_mode="values"
):
    print([(message.type, message.content) for message in event["messages"]])
```

```
for event in app.stream(
    {"messages": [{"role": "user", "content": "我叫什么名字"}]},
    config,
    stream_mode="values"
):
    print([(message.type, message.content) for message in event["messages"]])
```

输出如下，当请求完成时，如果消息数量>2，那么最早的两条消息会被删除。

```
[('human', '你好，我是ada')]
[('human', '你好，我是ada'), ('ai', '你好，Ada！很高兴认识你！😊\n\n我是DeepSeek，由深度求索公司创造的AI助手。我可以帮你解答各种问题，进行对话交流，协助处理文本任务等等。\n\n有什么我可以帮助你的吗？无论是学习、工作、生活方面的疑问，还是只是想聊聊天，我都很乐意陪伴你！✨')]
[('human', '你好，我是ada'), ('ai', '你好，Ada！很高兴认识你！😊\n\n我是DeepSeek，由深度求索公司创造的AI助手。我可以帮你解答各种问题，进行对话交流，协助处理文本任务等等。\n\n有什么我可以帮助你的吗？无论是学习、工作、生活方面的疑问，还是只是想聊聊天，我都很乐意陪伴你！✨'), ('human', '我叫什么名字')]
[('human', '你好，我是ada'), ('ai', '你好，Ada！很高兴认识你！😊\n\n我是DeepSeek，由深度求索公司创造的AI助手。我可以帮你解答各种问题，进行对话交流，协助处理文本任务等等。\n\n有什么我可以帮助你的吗？无论是学习、工作、生活方面的疑问，还是只是想聊聊天，我都很乐意陪伴你！✨'), ('human', '我叫什么名字'), ('ai', '你叫Ada！你刚刚告诉我“我是ada”，所以我记得你的名字是Ada。\n\n很高兴再次和你打招呼，Ada！有什么我可以帮你的吗？😊')]
[('human', '我叫什么名字'), ('ai', '你叫Ada！你刚刚告诉我“我是ada”，所以我记得你的名字是Ada。\n\n很高兴再次和你打招呼，Ada！有什么我可以帮你的吗？😊')]
```

清空所有消息：

```
from langgraph.graph.message import REMOVE_ALL_MESSAGES

def clear_messages(state):
    return {"messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES)]}

builder = StateGraph(MessagesState)
builder.add_sequence([call_model, clear_messages])
builder.add_edge(START, "call_model")

checkpointer = InMemorySaver()
app = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "1"}}

for event in app.stream(
    {"messages": [{"role": "user", "content": "你好，我是ada"}]},
    config,
    stream_mode="values"
):
    print([(message.type, message.content) for message in event["messages"]])

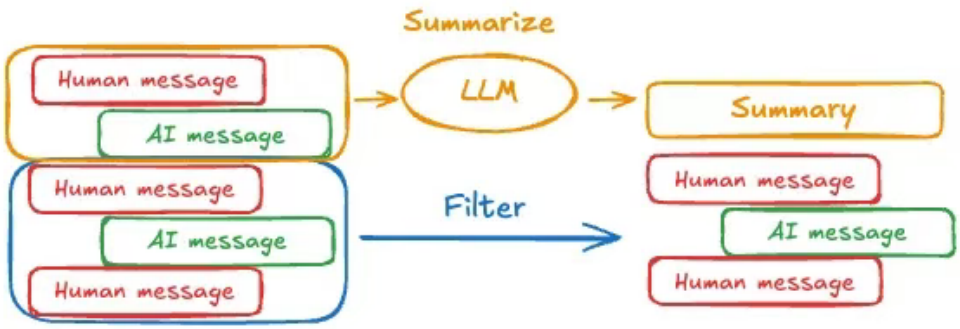
for event in app.stream(
    {"messages": [{"role": "user", "content": "我叫什么名字"}]},
    config,
    stream_mode="values"
):
    print([(message.type, message.content) for message in event["messages"]])
```

输出如下，请求完成后，会立即删除所有消息记录。

```
[('human', '你好，我是ada')]
[('human', '你好，我是ada'), ('ai', '你好，Ada！很高兴认识你！😊\n\n我是DeepSeek，由深度求索公司创造的AI助手。我可以帮你处理各种问题，比如：\n- 回答问题和解释概念\n- 协助写作和翻译\n- 提供学习建议\n- 帮助分析问题\n- 进行日常对话\n\n有什么我可以帮助你的吗？无论是工作学习还是生活上的问题，我都很乐意为你提供帮助！✨')]
[]
[('human', '我叫什么名字')]
[('human', '我叫什么名字'), ('ai', '很抱歉，我无法知道您的名字。我是人工智能助手，不具备获取用户个人信息的权限，包括您的名字。\n\n您可以在对话中直接告诉我您希望我如何称呼您，或者告诉我您的名字，这样我就可以在后续的对话中使用它了。例如，您可以说："你可以叫我小明"或者"我的名字是李华"。\n\n请问您希望我如何称呼您呢？')]
[]
```

总结消息

通过修剪、删除来管理历史消息，会有丢失信息的问题。为了避免这个问题，可以进行消息总结，也就是通过调用LLM对历史对话进行摘要，并将摘要作为新的上下文传入，以在减少消息数量的同时保留关键信息。



1.首先，安装LangMem，这是一个由LangChain维护的库，提供了用于在agent中管理记忆的工具。

```
pip install -U langmem
```

2.langmem库提供了一个预构建的**SummarizationNode**，可以极大地简化实现过程：

```
import tiktoken
from typing import Any, TypedDict

from langchain.chat_models import init_chat_model
from langchain_core.messages import AnyMessage, BaseMessage, SystemMessage
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langgraph.graph import StateGraph, START, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langmem.short_term import SummarizationNode, RunningSummary

summary_prompt = ChatPromptTemplate.from_messages(
    [
   MessagesPlaceholder(variable_name="messages"),
```

```
    # 使用 HumanMessage 模拟用户在最后发出总结指令
    ("human", "请根据以上对话，生成一段简洁、连贯的中文摘要，注意不要丢失细节"),
     ]
)

update_summary_prompt = ChatPromptTemplate.from_messages(
     [
    MessagesPlaceholder(variable_name="messages"),
    # 使用 HumanMessage 模拟用户在最后发出总结指令
    ("human", "以下是目前为止的对话摘要：{existing_summary}\n\n请根据以上新消息扩展此摘要："),
     ]
)

BASE_URL=""
TOKEN=""
MODEL_NAME=""

model = init_chat_model(
     model=MODEL_NAME,
     model_provider="openai",
     base_url=BASE_URL,
     api_key=TOKEN,
     temperature=0,
)
summarization_model = model.bind(max_tokens=128)

# count_tokens_approximately更适合英文分词，中文这里使用tiktoken来计算token数量
encoding = tiktoken.get_encoding("cl100k_base")

def count_tokens_accurately(messages: list[BaseMessage]) -> int:
     """使用 tiktoken 精确计算消息列表的 token 总数"""
     # 注意：langmem 的 token_counter 期望的输入是消息列表
     text_content = " ".join([msg.content for msg in messages if isinstance(msg.content, s
     return len(encoding.encode(text_content))

class State(MessagesState):
     context: dict[str, RunningSummary]

class LLMInputState(TypedDict):
     summarized_messages: list[AnyMessage]
     context: dict[str, RunningSummary]

summarization_node = SummarizationNode(
#token_counter=count_tokens_approximately,
     token_counter=count_tokens_accurately,  # 更换为自定义的token计算工具
     model=summarization_model,
     max_tokens=256,
     max_tokens_before_summary=256,
     max_summary_tokens=128,
```

```
    initial_summary_prompt=summary_prompt,  # 使用自定义prompt，默认为英文
    existing_summary_prompt=update_summary_prompt
)

def call_model(state: LLMInputState):
    response = model.invoke(state["summarized_messages"])
    return {"messages": [response]}

checkpointer = InMemorySaver()
builder = StateGraph(State)
builder.add_node(call_model)
builder.add_node("summarize", summarization_node)
builder.add_edge(START, "summarize")
builder.add_edge("summarize", "call_model")
graph = builder.compile(checkpointer=checkpointer)

# Invoke the graph
config = {"configurable": {"thread_id": "1"}}
graph.invoke({"messages": "你好，我叫ada"}, config)
graph.invoke({"messages": "请写一首诗，关于小狗的"}, config)
graph.invoke({"messages": "再写一首关于小猫的"}, config)
final_response = graph.invoke({"messages": "我叫什么名字？"}, config)

final_response["messages"][-1].pretty_print()
# 检查摘要是否生成
if"running_summary"in final_response["context"]:
    print("\n生成的摘要:", final_response["context"]["running_summary"].summary)
else:
    print("\n对话较短，尚未生成摘要。")
```

输出如下：

```
================================== Ai Message ==================================

你叫Ada。

生成的摘要：用户自我介绍说她叫Ada，并请求写一首关于小狗的诗。助手回应问候并写了一首描述小狗在阳光下欢快奔跑、充满喜悦和纯真爱的诗。随后，用户请求再写一首关于小猫的诗。助手回应并写了一首描述小猫轻盈如风、充满好奇心和温暖陪伴的诗。
```

如果需要在agent中实现的话，那么将SummarizationNode传入pre_model_hook即可。SummarizationNode会自动检查历史消息的长度，当token数量超过阈值时，触发一次总结，然后将”摘要 + 最新消息“的组合传递给model。其中，initial_summary_prompt用于第一次生成摘要时的prompt模板，existing_summary_prompt用于更新现有摘要的prompt模板，final_prompt是将摘要与剩余的消息合并后的prompt模板。

State中的context字段用于存储运行中的摘要信息，避免在每次调用时都重复总结。

检查点管理

对有状态的agent的记忆进行检查、管理和重置，对于监控agent和提高用户使用体验都必不可少，LangGraph提供了以下的一些工具，用来对检查点进行管理。

查看最近的短期记忆，也就是最近的检查点的状态：

```
graph.get_state(config=config)
```

查看线程的所有短期记忆，会按时间顺序返回这个线程所有的历史检查点：

```
graph.get_state_history(config=config)
```

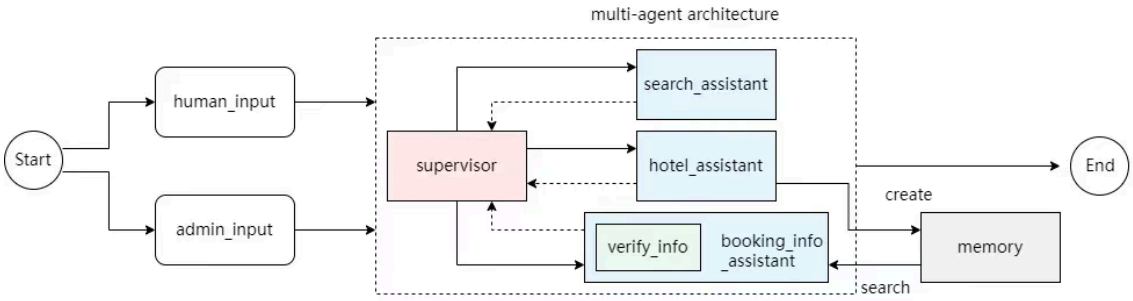
删除一个线程的所有短期记忆，一般用于重启对话场景：

```
checkpointer.delete_thread(thread_id)
```

引入MCP协议构建真实的Agent长记忆应用

本节将介绍如何基于Model Context Protocol（MCP）协议，使用LangGraph-Supervisor框架构建一个实用的、集成中断机制、有长记忆的多Agent系统。MCP是一种社区共建的AI开放协议，它标准化了应用向AI提供上下文的方式，极大简化了工具集成过程。

接下来，我们从头开始搭建multi-agent系统，模拟一个用户去进行旅游信息查询，并进行酒店预定，然后酒店管理侧可以查询用户的预定信息。整个Demo我们将展示Supervisor框架的搭建、人工介入、长短期记忆的应用等。



我们将逐步构建multi-agent工作流的每个组件，它包含三个子智能体，三个专门的 ReAct（推理和行动）子智能体，然后它们将组合起来创建一个包含额外步骤的多智能体工作流。

我们的工作流从以下开始：

1.human_input:用户输入;admin_input:管理员输入

2.supervisor协调三个子agent，根据input内容，选择合适的agent进行工作

3.当supervisor选择调用search_assistant的时候，那么查询信息,并将结果返回

4.当supervisor选择调用hotel_assistant的时候，那么把用户的预定信息,更新到Store中

5.当supervisor选择调用booking_info_assistant，会先进行verify_info，中断图的执行以请求管理员ID,当输入管理员ID后，接着判断管理员ID是否符合要求，如果不符合,那么不进行记忆查询，如果符合，则查询记忆,并返回。

步骤一：环境准备与安装

```
pip install langchain-mcp-adapters
```

步骤二：模型初始化

```
BASE_URL=""
TOKEN=""
MODEL_NAME=""

from langchain_openai import ChatOpenAI
from langchain.chat_models import init_chat_model

model = init_chat_model(
    model=MODEL_NAME,
    model_provider="openai",
    base_url=BASE_URL,
    api_key=TOKEN,
    temperature=0,
)
```

步骤三：初始化长短期记忆

```
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import InMemorySaver

store = InMemoryStore()
checkpointer = InMemorySaver()
```

步骤四：工具与助手配置

搜索助手

```
from typing import List
from typing_extensions import TypedDict

from langchain_core.messages import BaseMessage
from langchain_core.runnables import RunnableConfig
from langchain_mcp_adapters.client import MultiServerMCPClient
```

```
from langgraph.prebuilt import create_react_agent
from langgraph.config import get_store

from langgraph.graph import StateGraph, END
from langgraph.types import interrupt
from langchain_core.messages import AIMessage, ToolMessage, HumanMessage

# 搜索功能
url = ''
TOKEN = ''
search_client = MultiServerMCPClient(
    {
   "other_search": {
"url": url,
"headers": {
    "Authorization": f"Bearer {TOKEN}"
},
"transport": "sse"
   }
    }
)
search_tools = await search_client.get_tools()
search_agent = create_react_agent(
    model,
    search_tools,
    name="search_assistant",
    prompt="你是一个能搜索各种信息的助手。"
)
```




腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

酒店预定助手

定义图节点之间流动的共享数据结构，将需要存储的记忆格式化。

```
class UserInfo(TypedDict):
    user_id: str
    hotel_name: str
    date: str
    num_guests: int
```

定义预定酒店子智能体，并将用户预定历史存储下来。

```
def book_hotel(user_info: UserInfo, config: RunnableConfig):
    """处理酒店预订并更新长期记忆"""
    user_id = config["configurable"].get("user_id")
    print(user_info)

    hotel_name = user_info.get("hotel_name")
    date = user_info.get("date")
    num_guests = user_info.get("num_guests")

    # 存储用户个人预订历史
    namespace = ("user_bookings",)

    user_bookings = store.get(namespace, user_id) or []
    user_bookings.append(user_info)
    store.put(namespace, user_id, user_bookings)

    # 更新总预订计数
    namespace = ("total_hotel_bookings",)
    total_bookings = store.get(namespace, 'total_bookings_num') or 0
    store.put(namespace, 'total_bookings_num', total_bookings + 1)

    return f"成功为用户 {user_id} 预订了 {hotel_name}，入住日期：{date}，入住人数：{num_guests}

book_hotel_agent = create_react_agent(
    model=model,
    tools=[book_hotel],
    store=store,
    name="hotel_assistant",
    prompt="你是一个酒店预定助手。不需要用户ID、身份证号码、姓名和联系方式，就可以预定，请直接预定！"
)
```

查询助手

用户的预定信息都是需要保密的，只有特定的管理员才可以进行查询，因此，需要设计**中断机制，审核请求查询用户的权限是否符号要求**，即：在正式查询前，先中断一下，要求输入管理员ID信息；等输入后，接着执行图，再去判断管理员ID信息是否符合要求；只有符合才能正常进行用户信息查询。

1.查询工具定义：

```
def query_booking_from_store(config: RunnableConfig) -> str:
    """根据用户ID从存储中查询酒店预订信息。"""
    store = get_store()
```

```
    user_id = config["configurable"].get("user_id")
    booking_info = store.get(("user_bookings",), user_id)

    if booking_info and booking_info.value:
        return f"已找到预订信息：{str(booking_info.value)}"
    else:
        return "未找到该用户的预订信息"
```

2.定义子图的状态：

```
class SubgraphState(TypedDict):
    messages: List[BaseMessage]
```

3.创建新的图节点，负责中断和验证：

```
def authentication_and_query_node(state: SubgraphState, config: RunnableConfig):
    """
    这个节点首先中断图的执行以请求管理员ID，
    然后在恢复后验证ID，并调用工具查询信息。
    """
    # 核心：调用 interrupt() 来暂停图的执行
    admin_input = interrupt("请输入管理员id，如需退出查询，请输入exit")

    # 当图被恢复时，admin_input 将会获得传入的值
    if admin_input == "exit":
        result = "用户已退出查询。"
    elif admin_input == "admin_123":
        # 验证成功，调用真正的查询工具
        result = query_booking_from_store(config)
    else:
        # 验证失败
        result = f"没有权限查询：admin_id 不匹配（输入为：'{admin_input}'）"

    return {"messages": [AIMessage(content=result)]}
```

4.构建包含中断节点的子agent：

```
query_workflow = StateGraph(SubgraphState)
query_workflow.add_node("auth_and_query", authentication_and_query_node)
query_workflow.set_entry_point("auth_and_query")
query_workflow.add_edge("auth_and_query", END)
```

```
booking_query_subgraph = query_workflow.compile(checkpointer=checkpointer, store=store)

# 为子图命名，以便Supervisor可以调用它
booking_query_subgraph.name = "booking_info_assistant"
```

步骤五：Supervisor架构构建

基于上述组件，我们将构建一个完整的Supervisor架构工作流。

```
from langgraph_supervisor import create_supervisor

workflow = create_supervisor(
    [search_agent, book_hotel_agent, booking_query_subgraph],
    model=model,
    prompt=(
   "您是团队主管，负责管理信息搜索助手、酒店预订助手、以及用户信息查询助手。"
   "如需搜索各种信息，请交由 search_assistant 处理。"
   "如需预定酒店，请交由 hotel_assistant 处理。"
   "如需查询用户的酒店预定信息，请交由 booking_info_assistant 处理。"
   "**注意**，你每次只能调用一个助理agent！"
    ),
)

supervisor = workflow.compile(checkpointer=checkpointer, store=store)
```

步骤六：系统运行

第一次交互： 查询北京火锅店

```
config = {"configurable": {"thread_id": "1", "user_id": "user_123"}}

print("--- 第一次交互 ： 查询北京火锅店---")
async for chunk in supervisor.astream(
    {"messages": [("user", "北京最出名的老北京火锅是哪家？")]},
    config
):
    # 打印每个数据块的内容
    for key, value in chunk.items():
   print(f"Node: '{key}'")
   if value:
print("  value:")
print(value)
    print("----")
```

输出：

```
北京最出名的老北京火锅有很多家，其中比较有名的包括：\n\n1. **东来顺**：东来顺是北京最著名的老北京涮羊肉
```

第二次交互：根据上一步推荐的火锅店查询酒店

```
print("--- 第二次交互 ：根据上一步推荐的火锅店查询酒店---")
async for chunk in supervisor.astream(
    {"messages": [("user", "那第一个推荐的火锅店附近有哪些酒店呀")]},
    config
):
    # 打印每个数据块的内容
    for key, value in chunk.items():
   print(f"Node: '{key}'")
   if value:
print("  value:")
print(value)
    print("----")
```

输出：

```
东来顺火锅店位于北京市东城区东华门大街。以下是东来顺火锅店附近的一些酒店推荐：\n\n1. **北京王府井希尔顿
```

可以看到，supervisor记得上文中提到过的第一个推荐的火锅店，这是短期记忆的典型应用。

第三次交互：预定酒店

```
print("--- 第三次交互 ：预定酒店---")

async for chunk in supervisor.astream(
    {"messages": [("user", "帮我预定北京王府井希尔顿酒店酒店，预定日期：2025-11-13到2025-11-14，
    config
):
    for key, value in chunk.items():
   print(f"Node: '{key}'")
   if value:
print("  value:")
print(value)
    print("----")
```

输出：

```
已成功为您预订了北京王府井希尔顿酒店，入住日期为2025年11月13日至2025年11月14日，入住人数为1人。祝您旅
```

成功调用预定酒店助手，并完成酒店预定。

第四次交互：管理员查询预定信息

```
print("--- 第四次交互 ：管理员查询预定信息---")
config = {"configurable": {"thread_id": "2", "user_id": "user_123"}}  # 更换管理员操作线程

interrupt_data = None
interrupt_input = None
print("--- 第一次运行，将会触发中断 ---")
async for chunk in supervisor.astream(
    {"messages": [("user", "查询用户预定酒店信息")]},
    config,
):
    for key, value in chunk.items():
   print(f"Node: '{key}'")
   if value:
print("  value:")
print(value)

   if key == "__interrupt__":
print("\n======= 图已成功中断！=======")
interrupt_data = value[0]
print(f"中断信息: {interrupt_data.value}")
break

    if interrupt_data:
   break
    print("----")


if interrupt_data:
    # 模拟管理员输入正确的密码
    interrupt_input = Command(resume="admin_123")
    # 如果想测试错误的密码，可以使用下面这行
#interrupt_input = Command(resume="wrong_password")

    print(f"\n--- 接收到中断输入 '{interrupt_input}'，继续执行图 ---")
    # 恢复图的执行
    async for chunk in supervisor.astream(
   interrupt_input,
   config,
    ):
   for key, value in chunk.items():
print(f"Node: '{key}'")
if value:
```

```
    print("  value:")
    print(value)
print("----")
```

第一次运行，触发中断并输出：

```
请输入管理员id，如需退出查询，请输入exit
```

接着，程序模拟管理员输入正确的ID后，继续执行图，booking_info_assistant查询到长期记忆，并输出：

```
已找到预订信息：[{'user_id': 'user_123', 'hotel_name': '北京王府井希尔顿酒店', 'date': '2025-11
```

最终，supervisor返回最终的结果：

```
用户预定的酒店信息如下：\n- 用户ID：user_123\n- 酒店名称：北京王府井希尔顿酒店\n- 预定日期：2025-11
```

未来工作

更智能的记忆管理策略

在上述的示例（预定酒店）中，我们仅将用户预定信息直接进行存储、检索，但对于supervisor来说，记住和不同用户的过往交互是非常重要的，在后续的交互中，才能针对不同的用户，给出更合适的回应。让Agent能自主决定记忆的存储、遗忘、更新和检索优先级，才能真正模拟人类的记忆过程。因此，未来需要改进设计更智能的记忆管理策略。

记忆驱动的多智能体系统架构选择

在上述示例（预定酒店）中，我们选择了Supervisor架构进行实现，但这显然存在缺陷，管理员系统不应该和用户系统使用同一个中央智能体，当系统功能越来越完善时，这样的设计会使得supervisor非常繁杂、且难以维护，Supervisor架构更适合需要明确控制流程和集中决策的场景。融合记忆功能的Multi-Agent系统可以根据应用场景选择更合适的架构，例如Hierarchical架构，可以用于不同层级的记忆服务于不同目的（个体、团队、全局）的场景；Custom架构，预先定义好各个Agent的记忆走向，构建更灵活的系统。

腾讯小Q介绍

小Q是官方QQ助手，集成了腾讯混元、DeepSeek等前沿大模型技术，具备强大的多模态交互与复杂任务处理能力。

小Q

更懂QQ用户的官方AI助手



小Q 可以做什么？

AI写作
轻松实现内容创作

提供专业内容模版，覆盖周报/短视频脚本/小红书文案等场景。

Ark内容解析
升级消息理解能力

QQ频道帖子/图文h5/位置信息……跨场景内容实现一键转发，免去繁琐分享链路，直达问答重点。

支持拍照上传题目，快速解析学科问题，提供详细解题过程；拍食物图片测算卡路里，还能变身健康搭子。

AI识图
一键即搜，精准回答

AI画图
多种图片风格转换

支持图片生成，多种尺寸风格自由切换，可一键将图片变为美漫、乐高、毕加索、粘土、线条简笔画、像素等风格。

历史对话一目了然，任意话题随心跳转切换，并可根据日期和主题分组对话，随时翻查重要信息。

对话分组
多线畅聊，一键回溯

AI文件解析
秒读长文，信息处理更高效

支持腾讯文档及PDF/TXT/DOC/DOCX/PPT/PPTX多格式上传，秒级解析长文，快速生成精炼要点，即刻洞见信息核心。

支持实时录音、录音转写和智能总结，是课堂或培训中的学习搭子、职场会议中的速记助手。

AI纪要
实时听记，多语翻译

参考文献

[1] Chhikara P, Khant D, Aryan S, et al. Mem0: Building production-ready ai agents with scalable long-term memory[J]. arXiv preprint arXiv:2504.19413, 2025.

[2] **Langchain Docs**

福利时刻

抽奖助手

腾讯视频VIP月卡5份



小程序



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号



阅读 1.1万

留言

写留言

还没有留言，来写首评